

Contents

1	Terminal Management System	1
1.1	Project Layout	1
1.2	Environment	2
1.3	Docker Installation	2
1.4	Build	3
1.5	Run	3
1.6	Redis Caching	4
1.7	Database Initialization	4
1.8	Schema Note	4
1.9	Feature A Examples	5
1.10	Feature B Examples	5
1.11	Feature D Examples	5
1.12	Tests	5
1.13	CI/CD	6
1.14	Bonus Cron Cleanup	6
1.14.1	Cleanup File Details	7
1.14.2	Crontab Details	7
1.15	Data Schema Check	8

1 Terminal Management System

Author: Arvanitis Ioannis, Mellon Technologies employee

Flask API for the Mellon Group DevOps Bootcamp final assignment. The local stack runs with Docker Compose and includes:

- tms-api: Flask application built from app/Dockerfile
- mysql: official MySQL image with persistent data and automatic schema/seed import
- redis: official Redis image used as the cache layer
- tms-cleanup: crontab-based cleanup worker for expired decommissioned terminals

The Flask entrypoint is app/main.py. The cleanup worker uses a crontab entry executed by super-cronic inside a separate container.

1.1 Project Layout

```
.
├── app/
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── cleanup/
│   ├── Dockerfile
│   ├── cleanup.sh
│   ├── crontab
│   └── entrypoint.sh
├── db/
│   └── init/
│       ├── 01_schema.sql
│       └── 02_seed.sql
├── docker-compose.yml
├── .env.example
└── README.md
```

1.2 Environment

Create a local .env file from the template:

```
cp .env.example .env
```

Update the password values before running the stack. The real .env file is ignored by git.

1.3 Docker Installation

This project requires Docker Engine and the Docker Compose plugin. The commands below follow the official Docker installation flow for Ubuntu.

Remove conflicting distro packages if they exist:

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-docker containerd runc; do  
    sudo apt remove "$pkg"  
done
```

Install prerequisites and add Docker's official apt repository:

```
sudo apt update  
sudo apt install ca-certificates curl  
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF  
Types: deb  
URIs: https://download.docker.com/linux/ubuntu  
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")  
Components: stable  
Architectures: $(dpkg --print-architecture)  
Signed-By: /etc/apt/keyrings/docker.asc  
EOF
```

Install Docker Engine, Buildx, and Compose:

```
sudo apt update  
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Verify the installation:

```
sudo systemctl status docker  
docker --version  
docker compose version  
sudo docker run hello-world
```

If Docker is not running:

```
sudo systemctl start docker
```

Optional: run Docker without sudo by adding your user to the docker group:

```
sudo usermod -aG docker "$USER"  
newgrp docker  
docker run hello-world
```

Official references:

- Docker Engine on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
- Docker Compose plugin: <https://docs.docker.com/compose/install/linux/>

1.4 Build

Build all local images defined in `docker-compose.yml`:

```
docker compose build
```

Build only the Flask API image:

```
docker compose build tms-api
```

Build only the cleanup cron image:

```
docker compose build tms-cleanup
```

Build and start the full stack:

```
docker compose up --build
```

Run in the background:

```
docker compose up --build -d
```

Check running containers:

```
docker compose ps
```

Watch logs:

```
docker compose logs -f
docker compose logs -f tms-api
docker compose logs -f tms-cleanup
```

Stop the stack while preserving MySQL and Redis volumes:

```
docker compose down
```

Reset the database and re-run the SQL seed files from `db/init/`:

```
docker compose down -v
docker compose up --build
```

1.5 Run

```
docker compose up --build
```

The API will be available at:

```
http://localhost:5000
```

Useful endpoints:

- GET `/health`: checks both MySQL and Redis connectivity and returns 503 if either component is degraded
- GET `/schema/terminals`: runs the schema exploration query from the assignment
- GET `/terminals`: lists all terminals
- GET `/terminals?enabled=true`: lists enabled terminals
- GET `/terminals?enabled=false`: lists decommissioned/disabled terminals
- GET `/templates`: lists templates
- GET `/templates/<id>`: returns one template
- POST `/terminals/from-template`: creates a terminal from a template and merchant MID
- GET `/terminals/<tid>`: returns details for one terminal
- GET `/terminals/flagged`: lists terminals with a non-zero scenario number
- POST `/terminals/<tid>/flag`: sets `scenario_number` and updates `updated_on`

- POST /terminals/<tid>/unflag: sets scenario_number to 0 and updates updated_on
- POST /terminals/<tid>/decommission: disables a terminal and queues it for deletion after 3 days
- GET /terminals/decommissioned: lists terminals in the decommission queue
- GET /statistics/by-hardware: terminal count by hardware model
- GET /statistics/by-state: active/inactive terminal counts
- GET /statistics/by-hardware-family: terminal count by hardware family
- GET /statistics/idle-distribution: terminal count by idle-days bucket

Application logs are written to stdout with timestamp, level, and message. Database and Redis operations are wrapped with explicit error handling and return JSON error responses instead of silent failures.

1.6 Redis Caching

The API uses a cache-aside pattern with Redis. GET /terminals is cached for 30 seconds, and GET /statistics/* endpoints are cached for 60 seconds. Cache operations are best-effort: if Redis is unavailable, the API logs the Redis error and continues by reading from MySQL.

Every write endpoint clears the whole application cache before returning:

- POST /terminals/<tid>/flag
- POST /terminals/<tid>/unflag
- POST /terminals/<tid>/decommission
- POST /terminals/from-template

To verify cache behavior, watch the API logs:

```
docker compose logs -f tms-api
```

In another terminal:

```
curl http://localhost:5000/terminals
curl http://localhost:5000/terminals
curl -X POST http://localhost:5000/terminals/T0101001/flag \
-H "Content-Type: application/json" \
-d '{"scenario_number": "5"}'
curl http://localhost:5000/terminals
```

The expected log flow is Cache MISS, then Cache HIT, then cache clear after the write, then Cache MISS again.

1.7 Database Initialization

The SQL files in db/init/ are mounted to /docker-entrypoint-initdb.d inside the MySQL container. The official MySQL image executes them automatically only when the database volume is created for the first time.

To recreate the database from scratch:

```
docker compose down -v
docker compose up --build
```

1.8 Schema Note

The mid field belongs to the merchants table. The terminals table stores merchant_id, which references merchants.id; therefore a terminal's MID is retrieved through a join.

On API startup, the app applies the Feature A schema additions idempotently:

- Adds terminals.updated_on only if the column does not already exist.

- Creates decommission_queue only if the table does not already exist.

1.9 Feature A Examples

List terminals:

```
curl http://localhost:5000/terminals
curl "http://localhost:5000/terminals?enabled=true"
```

Terminal details:

```
curl http://localhost:5000/terminals/T0101001
```

Flag and unflag:

```
curl -X POST http://localhost:5000/terminals/T0101001/flag \
-H "Content-Type: application/json" \
-d '{"scenario_number": "5"}'

curl -X POST http://localhost:5000/terminals/T0101001/unflag
```

Decommission:

```
curl -X POST http://localhost:5000/terminals/T0101001/decommission
curl http://localhost:5000/terminals/decommissioned
```

1.10 Feature B Examples

List templates:

```
curl http://localhost:5000/templates
```

Template details:

```
curl http://localhost:5000/templates/1
```

Create terminal from template:

```
curl -X POST http://localhost:5000/terminals/from-template \
-H "Content-Type: application/json" \
-d '{"template_id": 1, "mid": "MID000101"}'
```

The create operation runs in a single database transaction. It validates the template, validates the merchant MID, locks the merchant's existing terminals, calculates the next TID suffix, inserts the new terminal, and returns 201 Created.

1.11 Feature D Examples

Statistics endpoints use Pandas for aggregation and Redis cache with a 60-second TTL:

```
curl http://localhost:5000/statistics/by-hardware
curl http://localhost:5000/statistics/by-state
curl http://localhost:5000/statistics/by-hardware-family
curl http://localhost:5000/statistics/idle-distribution
```

1.12 Tests

Run the unit tests with:

```
pytest -q
```

The tests use fake database and Redis objects, so they do not require Docker containers to be running.

1.13 CI/CD

GitHub Actions workflows live in `.github/workflows/`.

CI runs on pull requests, pushes to main or master, and manual runs from the GitHub Actions UI. It:

1. Checks out the repository.
2. Sets up Python 3.12.
3. Installs dependencies from `app/requirements.txt`.
4. Runs unit tests with `pytest -q`.
5. Creates `.env` from `.env.example`.
6. Validates docker compose config.
7. Builds the `tms-api` Docker image.
8. Builds the `tms-cleanup` Docker image.

CD runs on pushes to main and can also be started manually from the GitHub Actions UI. It:

1. Logs in to GitHub Container Registry with `GITHUB_TOKEN`.
2. Builds and pushes the API image:

```
ghcr.io/<owner>/tms-api:latest
ghcr.io/<owner>/tms-api:<commit-sha>
```

3. Builds and pushes the cleanup image:

```
ghcr.io/<owner>/tms-cleanup:latest
ghcr.io/<owner>/tms-cleanup:<commit-sha>
```

The CD workflow publishes images. A real server deployment step can be added later when the target host, SSH secrets, and deployment strategy are known.

Manual execution from the GitHub UI:

1. Open the repository on GitHub.
2. Go to Actions.
3. Select CI or CD.
4. Click Run workflow.
5. Select the branch and confirm Run workflow.

1.14 Bonus Cron Cleanup

The `tms-cleanup` service runs in a separate container with a crontab file. Configure the schedule with:

```
CLEANUP_CRON=0 2 * * *
```

At container startup, `cleanup/entrypoint.sh` writes this crontab line to `/etc/tms-cleanup.generated`:

```
0 2 * * * . /etc/tms-cleanup.env; /usr/local/bin/cleanup.sh >> /proc/1/fd/1 2>> /proc/1/fd/2
```

The shell script deletes expired rows where `decommission_queue.delete_after < NOW()`. Because `decommission_queue.tid` has a foreign key to `terminals.tid`, it deletes in this order inside one transaction:

1. Delete from `decommission_queue`.
2. Delete the matching rows from `terminals`.

Run the cleanup once manually for verification:

```
docker compose run --rm -e CLEANUP_ONCE=true tms-cleanup
```

Inspect the installed crontab file:

```
docker exec tms-cleanup cat /etc/tms-cleanup.generated
```

Watch cron runner logs:

```
docker compose logs -f tms-cleanup
```

1.14.1 Cleanup File Details

cleanup/Dockerfile

Builds the image for the cleanup container. It uses the official `mysql:8.4` image so the container has the correct MySQL 8 client and can authenticate against the MySQL server. It downloads `supercronic`, copies the cleanup scripts into the image, and sets `cleanup/entrypoint.sh` as the container entrypoint.

cleanup/entrypoint.sh

Runs when the `tms-cleanup` container starts. It creates `/etc/tms-cleanup.env` from Docker environment variables, creates `/etc/tms-cleanup.generated` with the final crontab line, prints that crontab line to stdout, and starts `supercronic`. If `CLEANUP_ONCE=true` is passed, it runs `cleanup.sh` once and exits instead of starting the cron runner.

cleanup/crontab

Contains the default crontab line:

```
0 2 * * * . /etc/tms-cleanup.env; /usr/local/bin/cleanup.sh >> /proc/1/fd/1 2>> /proc/1/fd/2
```

cleanup/cleanup.sh

Does the actual cleanup work. It validates required MySQL variables, selects expired TIDs from `decommission_queue`, deletes those rows from `decommission_queue`, then deletes the matching terminals from `terminals`. The delete order matters because `decommission_queue.tid` references `terminals.tid` with a foreign key. The script also clears Redis `tms:*` keys after a successful cleanup.

1.14.2 Crontab Details

The default schedule is:

```
0 2 * * *
```

Crontab fields are:

```
minute hour day-of-month month day-of-week
```

So `0 2 * * *` means:

- 0: at minute 0
- 2: at hour 2, meaning 02:00
- *: every day of the month
- *: every month
- *: every day of the week

Therefore the cleanup runs every day at 02:00 inside the cleanup container.

The full generated crontab command is:

```
0 2 * * * . /etc/tms-cleanup.env; /usr/local/bin/cleanup.sh >> /proc/1/fd/1 2>> /proc/1/fd/2
```

This does three things:

1. `. /etc/tms-cleanup.env` loads the MySQL and Redis environment variables for the cron process.
2. `/usr/local/bin/cleanup.sh` runs the cleanup script.
3. `>> /proc/1/fd/1 2>> /proc/1/fd/2` sends stdout and stderr to the container logs, so `docker compose logs tms-cleanup` shows cron output.

The generated environment file contains values like:

```
MYSQL_HOST='mysql'  
MYSQL_PORT='3306'  
MYSQL_DATABASE='tms'  
MYSQL_USER='tms_app'  
MYSQL_PASSWORD='...'  
REDIS_HOST='redis'  
REDIS_PORT='6379'  
REDIS_DB='0'
```

Inspect the generated files inside the running container:

```
docker exec tms-cleanup cat /etc/tms-cleanup.generated  
docker exec tms-cleanup cat /etc/tms-cleanup.env
```

1.15 Data Schema Check

Use this query to inspect the terminals table:

```
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE  
FROM INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'terminals'  
ORDER BY ORDINAL_POSITION;
```

After the API starts, the result includes the assignment seed columns plus the `updated_on` column added by the application migration.